



## PROYECTO FINAL

# I PORTADA

## UNIVERSIDAD TÉCNICA DE AMBATO

Facultad de Ingeniería en Sistemas, Electrónica e Industrial  
“Proyecto Académico de Fin de Semestre: Enero – Julio 2026”

|                                           |                                                                                                                                                   |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tema:</b>                              | Sistema inteligente de radar vehicular: medición de velocidad multi-vehículo, reconocimiento de placas ecuatorianas y sanción difusa proporcional |
| <b>Carrera:</b>                           | Ingeniería de Software                                                                                                                            |
| <b>Unidad de Organización Curricular:</b> | PROFESIONAL                                                                                                                                       |
| <b>Línea de Investigación:</b>            | Tecnologías de la Información y Sistemas de Control                                                                                               |
| <b>Nivel y Paralelo:</b>                  | Séptimo “A”                                                                                                                                       |
| <b>Alumnos participantes:</b>             | Paul Velastegui<br>David Manjarres                                                                                                                |
| <b>Asignatura:</b>                        | Inteligencia Artificial                                                                                                                           |
| <b>Docente:</b>                           | Ing. Rubén Nogales, Mg.                                                                                                                           |

# II INFORME DEL PROYECTO

## 0.1 Título

Sistema inteligente de radar vehicular para la medición simultánea de velocidad de múltiples vehículos, el reconocimiento de placas ecuatorianas y la evaluación difusa de sanciones proporcionales en un entorno universitario controlado.

## 0.2 Resumen

El proyecto implementa un radar vehicular inteligente orientado al control de velocidad en una zona universitaria. La solución procesa video en tiempo real o desde archivo y rastrea *simultáneamente* a todos los vehículos del cuadro mediante un detector YOLO11 acoplado al algoritmo de seguimiento ByteTrack, lo que permite medir la velocidad de entre uno y seis vehículos a la vez. La velocidad se estima cuando el vehículo cruza dos líneas virtuales separadas por una distancia conocida. Para cada vehículo cercano se detecta la placa con un modelo YOLO, se segmentan sus caracteres con una red U-Net y se clasifican individualmente con una CNN de 36 clases alfanuméricas; la lectura se estabiliza por votación temporal entre fotogramas y se valida con el formato ecuatoriano de tres letras y tres o cuatro dígitos. Los modelos de reconocimiento, entrenados originalmente en Keras, se convirtieron a formato



ONNX para ejecutarse sobre `onnxruntime` en Python 3.14 sin TensorFlow. La decisión disciplinaria se realiza con un sistema difuso de dos etapas: una clasificación Mamdani (felicitación / normal / multa) y una inferencia Takagi–Sugeno que convierte el exceso sobre el límite de 20 km/h en horas de indisponibilidad de forma suave, monótona y proporcional. El sistema registra los eventos, almacena evidencias visuales, notifica por correo electrónico y expone una interfaz web de monitoreo.

### 0.3 Palabras clave

Visión por computador, seguimiento multi-objeto, reconocimiento de placas, U-Net, CNN, YOLO, ByteTrack, lógica difusa, Mamdani, Takagi–Sugeno, ONNX, radar vehicular.

### 0.4 Objetivos

#### General:

- Desarrollar un sistema inteligente de radar vehicular que mida la velocidad de varios vehículos de forma simultánea, reconozca sus placas ecuatorianas y sugiera una sanción proporcional mediante lógica difusa, integrando todo en una aplicación web de monitoreo.

#### Específicos:

- Implementar el seguimiento multi-vehículo con YOLO11 y ByteTrack para medir la velocidad de cada vehículo al cruzar dos líneas virtuales calibradas.
- Reconocer la cadena alfanumérica de la placa combinando detección con YOLO, segmentación de caracteres con U-Net y clasificación con CNN, validando el formato vehicular ecuatoriano.
- Desplegar los modelos de reconocimiento mediante ONNX para ejecutarlos sin dependencia de TensorFlow, garantizando portabilidad y rendimiento.
- Diseñar un sistema de inferencia difusa que clasifique la conducta y calcule una sanción estrictamente proporcional al exceso de velocidad.
- Presentar los eventos, las evidencias y el razonamiento difuso en una interfaz web, con notificación automática por correo de las infracciones.



## 0.5 Modalidad

Presencial.

## 0.6 Listado de equipos, materiales y recursos

- Computador con GPU NVIDIA (aceleración CUDA para los detectores YOLO).
- Cámara web / teléfono como fuente de video; videos de tránsito del campus.
- Entorno de desarrollo Python 3.14.
- Internet y material del aula virtual de la asignatura.
- Herramientas para generar documentos en formato PDF.

## Dependencias de Python utilizadas

La Tabla 1 detalla cada dependencia, su función y el uso concreto que se le dio dentro del sistema. Todas son de código abierto.



Tabla 1: Dependencias del sistema: función y uso concreto.

| Dependencia                                 | Qué hace                                                                                      | Cómo se usó en el sistema                                                                                                                                                                              |
|---------------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ultralytics</code>                    | Implementación de modelos YOLO y del seguimiento multi-objeto (ByteTrack).                    | Carga <code>yolo11n</code> (COCO) para detectar y <i>rastrear</i> todos los vehículos del cuadro, y el modelo de placa ( <i>License_Plate</i> ) para localizar la región de la placa en cada vehículo. |
| <code>onnxruntime</code>                    | Motor de inferencia de modelos en formato ONNX, independiente del framework de entrenamiento. | Ejecuta en CPU la U-Net de segmentación de caracteres y la CNN de clasificación, convertidas desde Keras. Permite usar el OCR en Python 3.14 sin TensorFlow.                                           |
| <code>torch</code>                          | Backend de cómputo en GPU de los modelos YOLO.                                                | Acelera en CUDA la detección y el seguimiento de vehículos/placas; la GPU queda dedicada a YOLO mientras el OCR corre en CPU vía ONNX.                                                                 |
| <code>opencv-python</code>                  | Procesamiento de imagen y video clásico.                                                      | Captura de cámara/archivo, recorte y escalado de placas, enderezado por Hough, binarización y componentes conexos en el postproceso, y dibujo del HUD (líneas, cajas, etiquetas) sobre el video.       |
| <code>numpy</code>                          | Cálculo numérico vectorial sobre matrices.                                                    | Representa imágenes y máscaras, arma los <i>batches</i> de entrada a los modelos ONNX y realiza la votación y los cálculos del sistema difuso.                                                         |
| <code>scikit-fuzzy</code>                   | Funciones de pertenencia difusa (triangular, trapezoidal) e interpolación.                    | Define las pertenencias de velocidad y los sub-niveles de severidad, base de la clasificación Mamdani y de la sanción Takagi-Sugeno.                                                                   |
| <code>fastapi</code> / <code>uvicorn</code> | Framework y servidor web asíncrono.                                                           | Exponen la API REST (iniciar/detener, configurar fuente y distancia), transmiten el video procesado (MJPEG) y emiten los eventos al frontend.                                                          |
| <code>matplotlib</code>                     | Generación de figuras.                                                                        | Renderiza el razonamiento difuso (pertenencias de velocidad y curva de sanción) que se muestra en el modal del evento y se adjunta en el correo.                                                       |

### TAC (Tecnologías para el Aprendizaje y Conocimiento):

☐ Plataformas educativas    ☒ Aplicaciones educativas    ☒ Inteligencia Artificial    ☐ Recursos audiovisuales

## 0.7 Planteamiento del problema

La identificación automática de vehículos por su placa es una aplicación frecuente de la visión por computador en el control de accesos, la seguridad y el monitoreo de tránsito [1]. En entornos universitarios el control de velocidad es crítico para la protección peatonal, pero la vigilancia manual es costosa, no genera evidencia objetiva y no escala cuando circulan varios vehículos a la vez.

El problema abordado es, por tanto, construir un radar que, a partir de una sola cámara, sea capaz de: (i) medir la velocidad de *múltiples* vehículos simultáneamente; (ii) leer de forma confiable la placa ecuatoriana de cada uno pese a la distancia, el desenfoque por movimiento y la inclinación; y (iii) traducir el exceso de velocidad en una sanción *proporcional* e interpretable, evitando los saltos bruscos de los umbrales rígidos. Cada uno de estos puntos plantea un reto técnico distinto: el seguimiento estable de

identidades, el reconocimiento robusto de caracteres y la toma de decisiones bajo variables graduales.

## 0.8 Marco de referencia: formato de placas ecuatorianas

El sistema considera placas ecuatorianas de seis o siete caracteres: tres letras iniciales seguidas de tres o cuatro dígitos (p. ej. ABC-1234). Esta estructura se aprovecha como una capa de validación que descarta lecturas anómalas generadas por ruido visual o regiones que no corresponden a una placa, y permite aceptar tanto el modelo actual como el antiguo de placa ecuatoriana.

## 0.9 Metodología

### 0.9.1 Arquitectura general del sistema

El sistema se organiza en cuatro bloques: adquisición y seguimiento de video, pipeline de visión por computador, decisión difusa y backend/frontend de monitoreo. La Figura 1 resume el flujo completo de un evento.

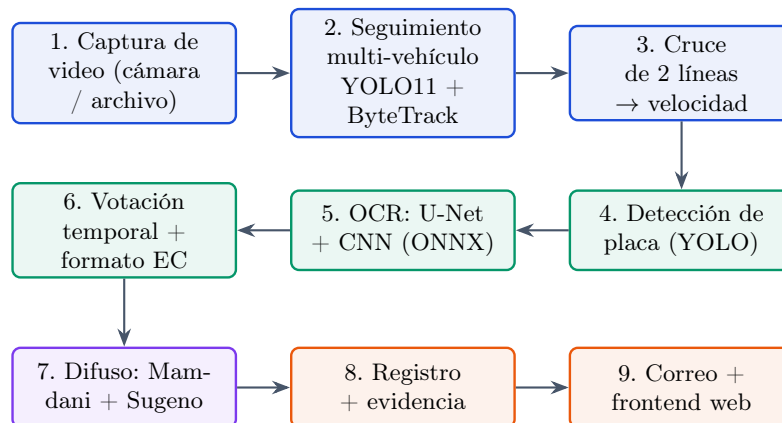


Figura 1: Flujo general del radar vehicular: del video al evento registrado.

Fuente: Elaboración propia.

### 0.9.2 Seguimiento multi-vehículo y medición de velocidad

A diferencia de un enfoque de un solo vehículo, el sistema rastrea *todos* los vehículos del cuadro. Cada fotograma se reduce al 50 % y se procesa con un detector YOLO11 en su variante *nano* (clases COCO: automóvil, motocicleta, bus y camión) [2], acoplado al algoritmo de seguimiento ByteTrack, que asocia cada detección con una identidad estable entre fotogramas [3]. Gracias a estas identidades, el sistema mantiene un estado independiente por vehículo (cruces, tiempos, lecturas de placa), lo que habilita la medición simultánea de entre uno y seis vehículos.

La zona de medición se define con dos líneas virtuales separadas por una distancia conocida  $d$ . Para cada vehículo se observa el punto de contacto con el suelo (centro inferior de su caja). Cuando ese punto



cruza la primera línea se registra el tiempo  $t_a$  y cuando cruza la segunda,  $t_b$ . La velocidad se calcula como:

$$v = \frac{d}{|t_b - t_a|} \times 3,6 \quad [\text{km/h}], \quad (1)$$

donde el factor 3,6 convierte de m/s a km/h. Para fuentes de archivo se emplean las marcas de tiempo del propio video (no el reloj de CPU), de modo que la medición es exacta aunque el procesamiento sea más lento que el tiempo real.

**Criterio de muestreo (Nyquist).** La confiabilidad de la medición depende del número de fotogramas observados durante el cruce,  $N = dt \cdot F_s$ , con  $F_s$  el fps efectivo. Con pocos fotogramas el sistema dispone de poca evidencia temporal y la velocidad puede inflarse; con más fotogramas el error relativo disminuye como  $\approx 1/N$ . Esta es la versión temporal del criterio de muestreo: la frecuencia debe ser suficiente para capturar el fenómeno de interés [4]. De forma análoga, en el dominio espacial la resolución determina cuántos píxeles cubren la placa; una placa lejana se reduce a muy pocos píxeles y pierde legibilidad.

### 0.9.3 Detección de la placa

La región de la placa se localiza con un modelo YOLO entrenado para una sola clase (*License\_Plate*). YOLO se eligió por su detección en una sola etapa, adecuada para tiempo real [2]. La red redimensiona internamente la entrada a  $416 \times 416$ , pero el recorte de la placa se extrae del fotograma original en su resolución completa; así las etapas posteriores de segmentación y clasificación disponen del máximo detalle posible. Cuando la placa recortada es pequeña se amplía por interpolación cúbica antes del OCR.

### 0.9.4 Reconocimiento de la placa: U-Net + CNN

El reconocimiento se realiza en tres pasos (Figura 2). Primero, el recorte de la placa se endereza por rotación estimando la inclinación con la transformada de Hough (corrección puramente geométrica, sin dependencias adicionales). Segundo, una red U-Net [6] de entrada  $96 \times 256 \times 1$  produce una máscara probabilística donde cada píxel indica su probabilidad de pertenecer a un carácter; la máscara se binariza, se extraen contornos y se generan recortes individuales por carácter. La U-Net *no* clasifica caracteres: su única función es separar a nivel de píxel las regiones de tinta, lo que generaliza mucho mejor que una umbralización fija ante placas variadas, sombras o inclinación. Tercero, cada recorte se redimensiona a  $64 \times 64$  y se clasifica con una CNN de 36 clases (dígitos 0–9 y letras A–Z) de filosofía VGG con *Global Average Pooling* [7].

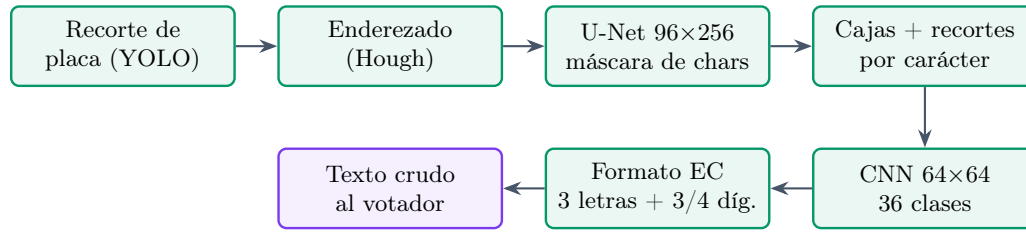


Figura 2: Cadena de reconocimiento de la placa: detección, enderezado, segmentación U-Net, clasificación CNN y validación de formato.

Fuente: Elaboración propia.

**Despliegue con ONNX (sin TensorFlow).** Los pesos de la U-Net y de la CNN se entrenaron originalmente en Keras (formato `.keras`). Dado que el entorno de ejecución es Python 3.14 —donde TensorFlow no dispone de paquetes— los modelos se convirtieron una sola vez a formato ONNX y se ejecutan con `onnxruntime` [13]. Esta decisión elimina la dependencia de TensorFlow, evita el conflicto de memoria de GPU con PyTorch (usado por YOLO) y mantiene la inferencia del OCR en CPU, liberando la GPU para el seguimiento.

### 0.9.5 Validación de formato y votación temporal

Como la misma placa se observa en muchos fotogramas mientras el vehículo cruza la zona, el sistema acumula las lecturas crudas por vehículo y produce un *consenso* por votación posición a posición ponderada por confianza y por tamaño de la placa (los fotogramas cercanos pesan más). Sobre ese consenso se aplica la validación de formato ecuatoriano; sólo se registra un evento cuando el consenso cumple la estructura 3 letras + 3/4 dígitos. La votación corrige los errores de un solo fotograma (desenfoque, ángulo) y eleva notablemente la precisión final.

### 0.9.6 Sistema de inferencia difusa

La decisión disciplinaria se modela con lógica difusa, adecuada cuando intervienen variables graduales como la velocidad, en lugar de umbrales rígidos [14]. El diseño tiene dos etapas (Figura 3):

1. **Clasificación (Mamdani).** Tres conjuntos difusos de velocidad —*felicitación* (0–10), *normal* (10–20) y *multa* (>20)— determinan la categoría por máxima pertenencia [15]. La pertenencia de *multa* es una rampa monótona desde el límite de 20 km/h, lo que evita el perfil en “serrucho” que produciría el máximo de varios triángulos solapados.
2. **Sanción (Takagi–Sugeno).** Sólo cuando la categoría es *multa*, las horas de indisponibilidad se obtienen con una inferencia Sugeno de orden cero [16]: cinco sub-niveles de severidad (leve...extrema) que teselan el rango 20–40 km/h, cada uno con un consecuente *singleton*

creciente. La salida es el promedio ponderado

$$H(v) = \frac{\sum_i \mu_i(v) c_i}{\sum_i \mu_i(v)}, \quad (2)$$

donde  $\mu_i(v)$  es la pertenencia al sub-nivel  $i$  y  $c_i$  su singleton en horas.

El motivo de usar Sugeno para la sanción es técnico: el centroide de Mamdani introduce mesetas y micro-descensos no monótonos cerca de los bordes de cada conjunto, mientras que el promedio ponderado de singletons (2) produce una salida *estrictamente monótona y suave*, que es precisamente la proporcionalidad requerida —a mayor exceso sobre 20 km/h, mayor sanción—. Las funciones de pertenencia empleadas son triangulares  $\mu_{tri}(x; a, b, c)$  y trapezoidales  $\mu_{trap}(x; a, b, c, d)$  estándar.

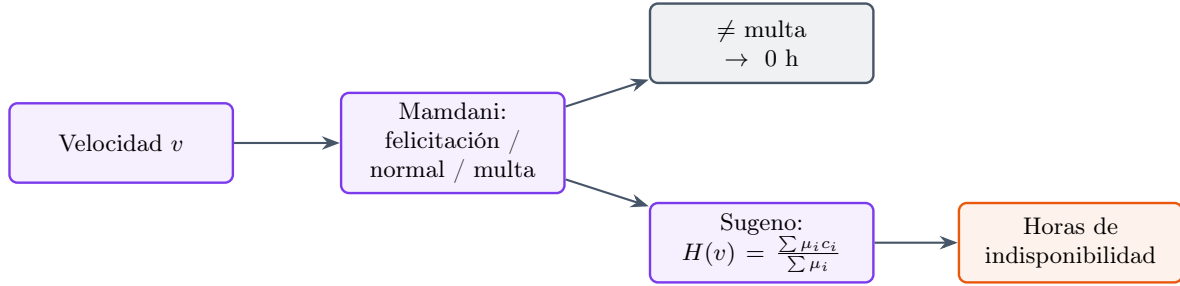


Figura 3: Decisión difusa en dos etapas: clasificación Mamdani y sanción proporcional Takagi–Sugeno.

Fuente: Elaboración propia.

### 0.9.7 Backend, frontend y notificación

El backend, construido con FastAPI, integra el pipeline de visión, transmite el video procesado y emite eventos. Por cada evento guarda el fotograma con la placa y la velocidad, registra el evento y dispara una notificación por correo con la evidencia y el razonamiento difuso. El frontend permite seleccionar la fuente, arrastrar las líneas de medición, observar el video en vivo, revisar el historial de eventos con miniaturas y visualizar el detalle de cada evento.

### 0.10 Conjuntos de datos y entrenamiento

La trazabilidad del origen de los datos es parte central del informe. Los tres modelos de reconocimiento se entrenaron con conjuntos públicos alojados en Roboflow Universe, todos bajo licencia CC BY 4.0.



### 0.10.1 Detección de placa

El detector de placa se entrenó con el conjunto público *License Plate Recognition* (v13) de Roboflow Universe, de origen internacional y una sola clase *License\_Plate*, con 101 866 imágenes generadas por aumento de datos a partir de  $\sim 7\,276$  imágenes fuente [5]. El reparto se muestra en la Tabla 2. Sobre estos datos se entrenó una red YOLO en variante *nano*, con entrada  $416 \times 416$ . El uso de un conjunto amplio busca que el detector aprenda la apariencia general de una placa; la validación de formato adapta la lectura final al contexto ecuatoriano.

Tabla 2: Conjunto de detección de placas (Roboflow, v13) [5].

| Partición     | Número de imágenes |
|---------------|--------------------|
| Entrenamiento | 98 798             |
| Validación    | 2 048              |
| Prueba        | 1 020              |
| Total         | 101 866            |

### 0.10.2 Clasificación de caracteres (CNN)

El clasificador CNN se entrenó con recortes alfanuméricos provenientes de un conjunto de placas del Reino Unido [9] y un conjunto de placas de Brasil [10], sobre una entrada de  $64 \times 64$  en escala de grises y 36 clases. El entrenamiento empleó aumento de datos, pesos por clase y pérdida focal [8] para compensar el desbalance natural entre caracteres frecuentes y poco frecuentes. Pruebas exploratorias confirmaron que  $64 \times 64$  preserva mejor los rasgos discriminantes que  $32 \times 32$  o  $48 \times 48$  (Tabla 5).

### 0.10.3 Segmentación de caracteres (U-Net)

Para la U-Net se combinaron un conjunto base de la India, recortes reales de placas ecuatorianas obtenidos del dataset de detección ecuatoriano [12], y subconjuntos de Brasil [10] y Reino Unido [9], además de un conjunto específico de segmentación de caracteres [11]. La función de pérdida combinó entropía cruzada binaria ponderada y pérdida Dice, con un mapa de pesos que penaliza los separadores entre caracteres vecinos para evitar que dos caracteres se fusionen en un mismo recorte.

## 0.11 Resultados y discusión



### 0.11.1 Detección de placa

El detector YOLO alcanzó los valores de la Tabla 3. Una precisión de 0,990 y un mAP@50 de 0,950 indican que la región de la placa se localiza de forma confiable, condición necesaria para que la segmentación y la clasificación operen sobre un recorte adecuado. El mAP@50–95 de 0,661, más bajo por su exigencia de ajuste fino de la caja, es suficiente para el recorte de placa, ya que la segmentación posterior no requiere un ajuste perfecto de bordes.

Tabla 3: Métricas del detector YOLO de placas.

| Métrica                         | Valor |
|---------------------------------|-------|
| Precisión                       | 0.990 |
| Exhaustividad ( <i>recall</i> ) | 0.924 |
| mAP@50                          | 0.950 |
| mAP@50–95                       | 0.661 |

### 0.11.2 Clasificación de caracteres

El clasificador CNN obtuvo una exactitud global de 0,9132 sobre un conjunto de prueba independiente, con métricas macro y ponderadas superiores a 0,90 (Tabla 4). Las clases con menor desempeño corresponden a caracteres visualmente ambiguos (p. ej. O/0, I/1), coherente con la dificultad del problema. La comparación de resoluciones de entrada (Tabla 5) justifica la elección de  $64 \times 64$ .

Tabla 4: Métricas globales del clasificador CNN de caracteres.

| Métrica            | Precisión | Recall | F1     |
|--------------------|-----------|--------|--------|
| Promedio macro     | 0.9140    | 0.9040 | 0.9075 |
| Promedio ponderado | 0.9143    | 0.9132 | 0.9129 |
| Exactitud global   | 0.9132    |        |        |

Tabla 5: Comparación de resolución de entrada para la clasificación.

| Resolución     | Exactitud | Observación                                  |
|----------------|-----------|----------------------------------------------|
| $32 \times 32$ | 0.81      | Mayor pérdida de detalle en trazos finos     |
| $48 \times 48$ | 0.87      | Mejora parcial, aún con confusiones visuales |
| $64 \times 64$ | 0.9132    | Mejor preservación de rasgos discriminantes  |



### 0.11.3 Segmentación

La U-Net se evaluó con el coeficiente Dice e IoU binario, métricas apropiadas para segmentación de máscaras, alcanzando un Dice cercano a 0,91. A diferencia de la clasificación, aquí no corresponde una matriz de confusión multiclase porque la salida es una máscara binaria de carácter/fondo. La segmentación aprendida aporta robustez frente a inclinación e iluminación, lo que se traduce en una mejor generalización a placas no vistas durante el entrenamiento.

### 0.11.4 Funcionamiento del radar en eventos reales

La aplicación integra las etapas de video, medición de velocidad, lectura de placa, cálculo de sanción y visualización. La Figura 4 muestra el dashboard de monitoreo en tiempo real, con las dos líneas de medición, el panel de configuración de la fuente, la última placa leída con su velocidad y el historial de eventos con miniaturas.

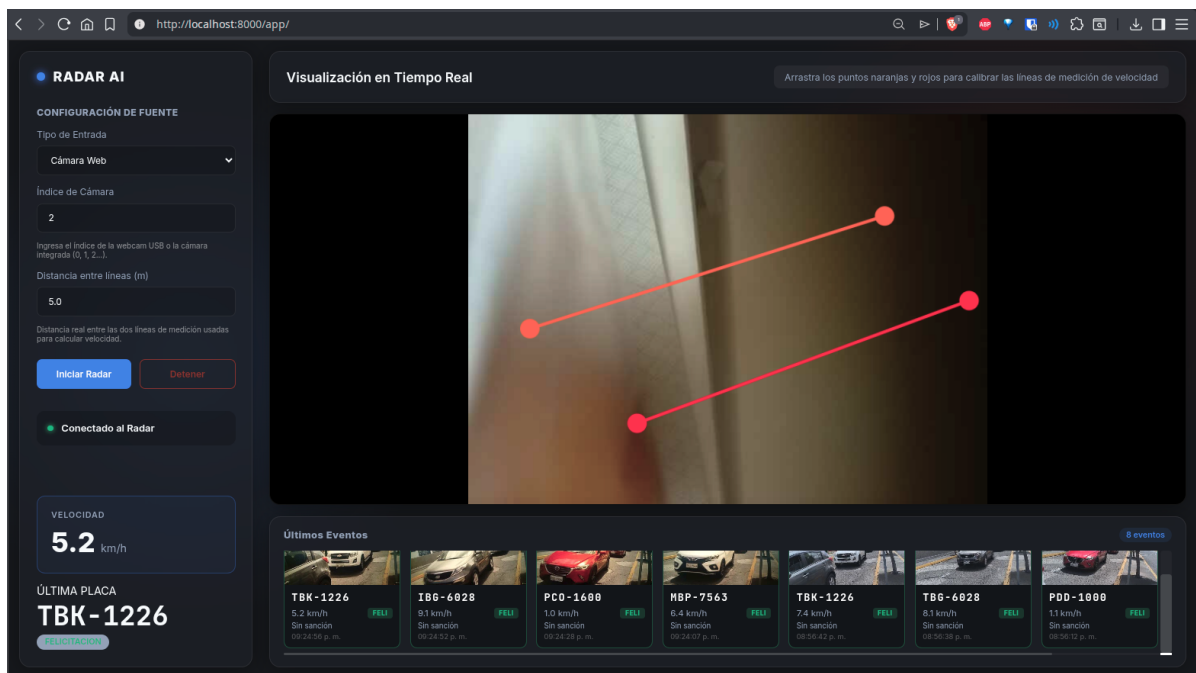


Figura 4: Dashboard de monitoreo en tiempo real con las líneas de medición, el panel de fuente y el historial de eventos.

Fuente: Elaboración propia.

En una evaluación sobre un video de tránsito del campus, el radar midió la velocidad de cuatro vehículos y registró los cuatro con su placa leída, de forma repetible entre ejecuciones, demostrando la operación multi-vehículo. La Figura 5 presenta el detalle de un evento: el fotograma capturado, la placa, la velocidad, la clasificación difusa y el razonamiento asociado.

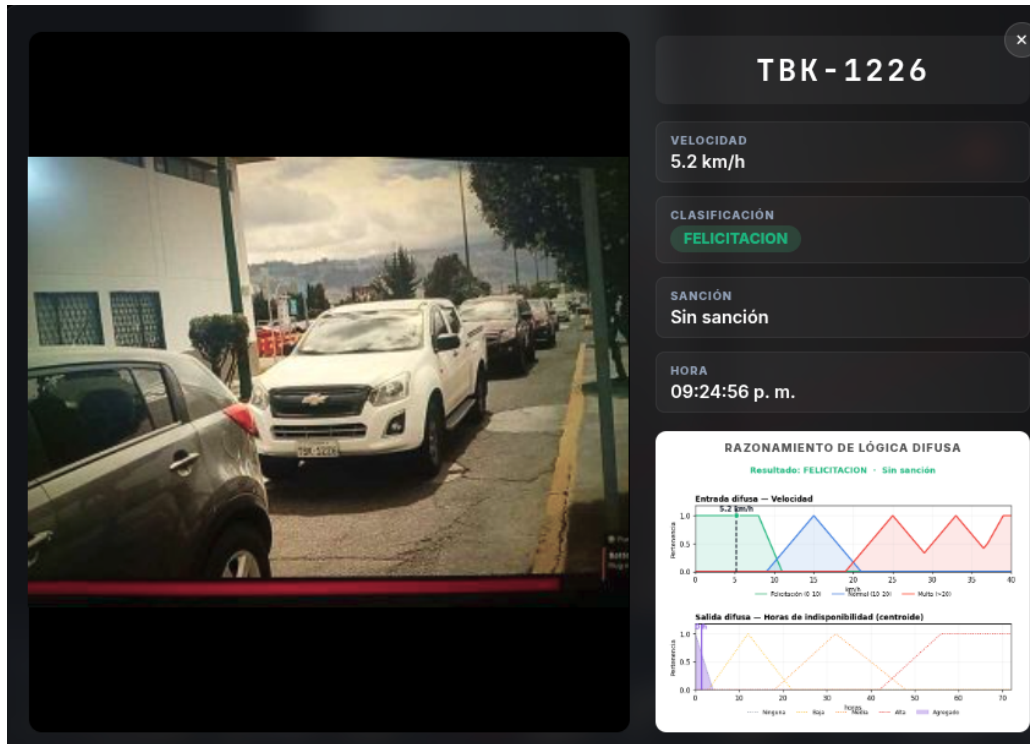


Figura 5: Detalle de un evento: placa TBK-1226, velocidad 5.2 km/h, clasificación *felicitación* (sin sanción) y razonamiento difuso.

Fuente: Elaboración propia.

#### 0.11.5 Sanción difusa proporcional

La Figura 6 muestra el razonamiento difuso para un caso de multa: el panel superior contiene las pertenencias de velocidad con la categoría *multa* como rampa limpia, y el panel inferior la curva de sanción  $H(v)$ , que es nula en la zona permitida ( $\leq 20$  km/h) y crece de forma suave y monótona. La Tabla 6 confirma la proporcionalidad: la sanción aumenta estrictamente con la velocidad, desde cero en el límite hasta cerca del máximo de tres días a 40 km/h.

**Resultado: MULTA · 1 día con 2 horas**

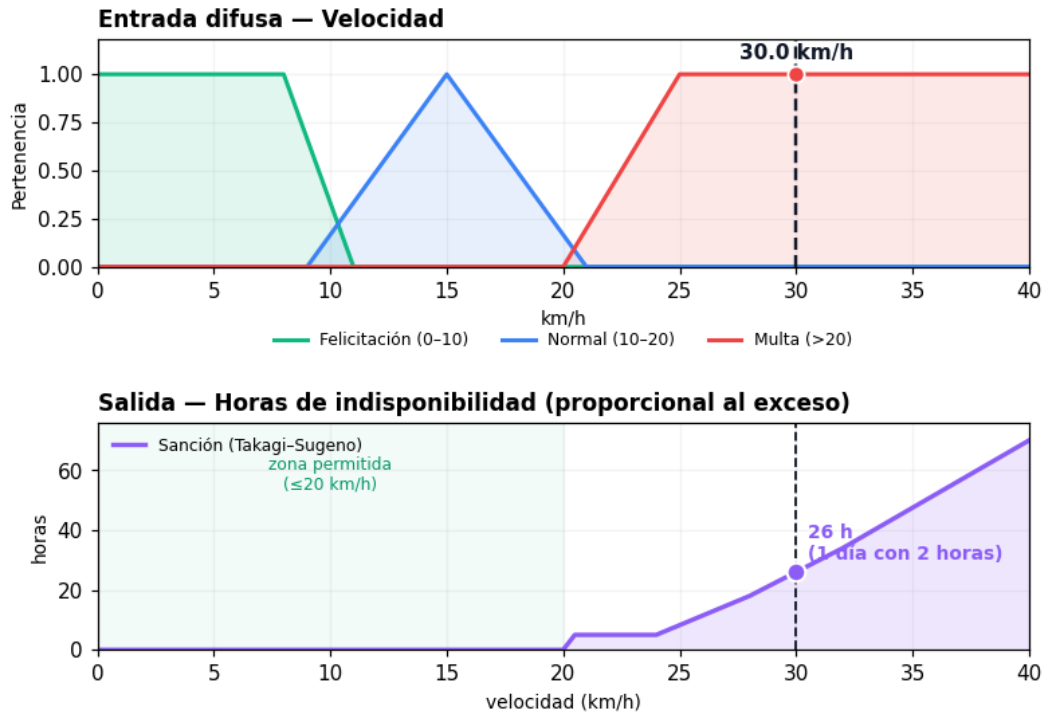


Figura 6: Razonamiento difuso para un evento de multa a 30 km/h: pertenencias de velocidad (arriba) y curva de sanción proporcional  $H(v)$  (abajo).

Fuente: Elaboración propia.

Tabla 6: Sanción sugerida según la velocidad (salida Takagi–Sugeno).

| Velocidad      | Horas | Equivalente                  |
|----------------|-------|------------------------------|
| $\leq 20$ km/h | 0     | Sin sanción (zona permitida) |
| 21 km/h        | 5     | 5 horas                      |
| 25 km/h        | 8     | 8 horas                      |
| 30 km/h        | 26    | $\approx 1$ día              |
| 35 km/h        | 48    | 2 días                       |
| 40 km/h        | 70    | $\approx 3$ días             |

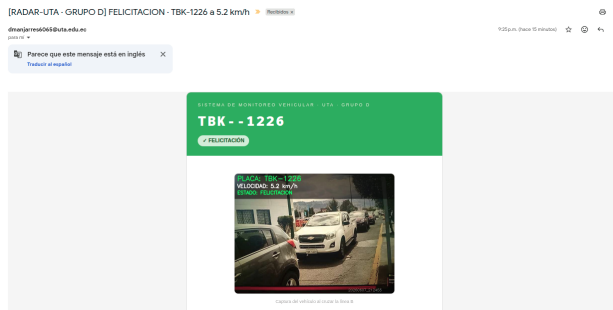
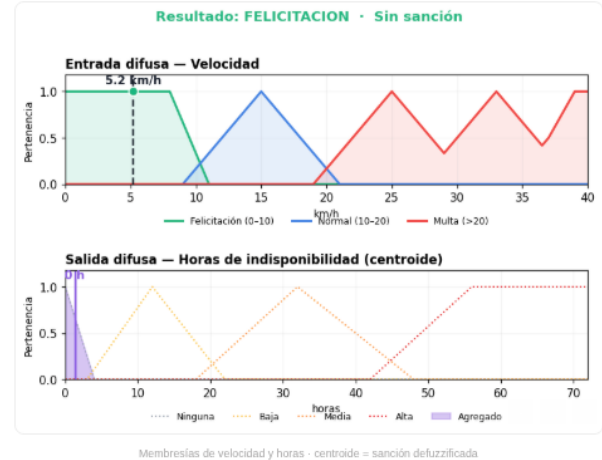
#### 0.11.6 Notificación por correo

Cada infracción genera un correo automático con la evidencia del evento, la placa, la velocidad, la clasificación y el razonamiento difuso, como muestran las Figuras 7 y 7. Esto convierte la detección en un registro accionable y trazable.

#### DETALLES DEL EVENTO

|                      |                                          |
|----------------------|------------------------------------------|
| Velocidad Registrada | 5.2 km/h                                 |
| Clasificación Difusa | ✓ FELICITACIÓN                           |
| Rango de Velocidad   | 0 – 10 km/h (Felicitación - zona segura) |
| Sanción              | Sin sanción                              |

#### RAZONAMIENTO DE LÓGICA DIFUSA



(a)

**RADAR AI · UTA** Grupo D — Sistema Inteligente de Monitoreo Vehicular  
 Manjarres Quintero, David Oswaldo · Velastegui Eugenio, Anthony Paul

(b)

Figura 7: Correos de notificación generados automáticamente ante una infracción, con evidencia visual y datos del evento.

Fuente: Elaboración propia.

### 0.11.7 Discusión

Los resultados muestran que la solución integra adecuadamente la visión por computador y la decisión difusa. El seguimiento con ByteTrack permite el caso multi-vehículo sin confundir identidades, y la votación temporal eleva la confiabilidad de la lectura por encima de la de un solo fotograma. La segmentación con U-Net aporta robustez frente a placas variadas, aunque su desempeño final sigue condicionado por la calidad del recorte, la iluminación y la inclinación. En el módulo difuso, el uso de Takagi-Sugeno para la sanción garantiza una salida monótona y proporcional, superando las metas y micro-descensos del centroide de Mamdani. Finalmente, el despliegue con ONNX demuestra que es posible reutilizar modelos entrenados en Keras dentro de un entorno moderno sin TensorFlow, manteniendo el rendimiento y separando el uso de GPU (seguimiento) del de CPU (reconocimiento).



## 0.12 Alcance, generalización y limitaciones

Una pregunta central es si el sistema funciona únicamente sobre los datos con los que se entrenó (placas, caracteres y videos específicos) o si *generaliza* a casos nuevos. La evidencia y el diseño indican que el sistema generaliza, con condiciones:

- **Detección y seguimiento de vehículos.** El modelo `yolo11n` está preentrenado sobre COCO (un conjunto general de 80 categorías que incluye automóvil, motocicleta, bus y camión). No se entrenó con nuestros videos, por lo que detecta y rastrea vehículos en cualquier escena o cámara.
- **Detección de placa.** El detector se entrenó con un conjunto internacional de  $\sim 101\,866$  imágenes de placas [5], no con nuestro video. En pruebas a distintas escalas localizó la placa de forma consistente, lo que confirma que aprende la apariencia general de una placa y no casos memorizados.
- **Reconocimiento de caracteres.** La U-Net y la CNN se entrenaron con placas de Reino Unido, Brasil e India y recortes ecuatorianos [9, 10, 11, 12]. El hecho de que lean placas ecuatorianas de campus —ausentes del entrenamiento— demuestra generalización; la votación temporal entre fotogramas corrige los errores puntuales de un solo cuadro.

En consecuencia, el sistema *no* está limitado al video usado en las pruebas: opera sobre cualquier cámara, vehículo y placa. No obstante, su alcance y desempeño están acotados por:

- **Formato ecuatoriano.** La validación final acepta solo la estructura de tres letras y tres o cuatro dígitos; placas de otros países se descartan de forma intencional, pues el sistema está dirigido al contexto nacional.
- **Condiciones de captura.** La lectura es confiable cuando la placa se observa con suficientes píxeles (vehículo cercano), de forma razonablemente frontal y sin desenfoque extremo, suciedad u oclusión; placas muy lejanas o muy borrosas reducen la exactitud, según el límite de muestreo espacial ya descrito.
- **Calibración de la velocidad.** La exactitud de la velocidad depende de una distancia entre líneas correctamente medida, su posición acorde a la escena y un fps efectivo suficiente.
- **Confusiones residuales.** Persisten confusiones entre caracteres visualmente similares (O/0, I/1), mitigadas —no eliminadas— por la votación.

En síntesis, para un despliegue nuevo basta con apuntar la cámara, calibrar las dos líneas a la distancia real y asegurar que las placas se vean legibles; no se requiere reentrenar ningún modelo.



### 0.13 Habilidades blandas empleadas

■ Trabajo en equipo   ■ Responsabilidad   ■ Pensamiento crítico   ■ Resolución de problemas

### 0.14 Conclusiones

- Se desarrolló un radar vehicular funcional que mide la velocidad de varios vehículos de forma simultánea mediante YOLO11 y ByteTrack, y registra cada uno con su placa y su sanción.
- El reconocimiento de placas en cascada (detección YOLO, segmentación U-Net y clasificación CNN) con validación de formato ecuatoriano y votación temporal logra lecturas confiables sobre placas variadas, no solo las vistas en entrenamiento.
- La conversión de los modelos a ONNX permitió ejecutar el OCR en Python 3.14 sin TensorFlow, eliminando conflictos de dependencias y de memoria de GPU.
- El sistema difuso de dos etapas (clasificación Mamdani + sanción Sugeno) produce una sanción estrictamente monótona y proporcional al exceso sobre 20 km/h, corrigiendo el comportamiento no monótono de un esquema basado únicamente en el centroide de Mamdani.
- La arquitectura modular y la interfaz web con evidencia por etapa facilitan auditar el origen de cualquier error: detección, segmentación, clasificación, validación o decisión difusa.

### 0.15 Recomendaciones

- Calibrar la distancia entre líneas y su posición según la geometría real de la cámara; un fps efectivo bajo o una distancia mal medida degradan la exactitud de la velocidad.
- Para máxima legibilidad de placas, ubicar la zona de medición donde el vehículo se observe grande y de frente; las placas lejanas pierden píxeles y legibilidad por el límite de muestreo espacial.
- Como trabajo futuro, reentrenar el clasificador con más recortes reales de placas ecuatorianas para reducir las confusiones entre caracteres similares, y explorar un detector de caracteres de extremo a extremo que evite la etapa de segmentación.



## 0.16 Referencias bibliográficas

- [1] J. Shashirangana, H. Padmasiri, D. Meedeniya, and C. Perera, “Automated license plate recognition: A survey on methods and techniques,” *IEEE Access*, vol. 9, pp. 11 203–11 225, 2021.
- [2] G. Jocher and J. Qiu, “Ultralytics YOLO11,” Ultralytics, 2024. [En línea]. Disponible: <https://docs.ultralytics.com/models/yolo11/>
- [3] Y. Zhang *et al.*, “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. European Conf. Computer Vision (ECCV)*, 2022, pp. 1–21.
- [4] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Cham, Switzerland: Springer, 2022.
- [5] Roboflow Universe Projects, “License Plate Recognition Dataset (v13),” Roboflow Universe, CC BY 4.0, 2026. [En línea]. Disponible: <https://universe.roboflow.com/roboflow-universe-projects/license-plate-recognition-rxg4e/dataset/13>
- [6] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [7] M. Salemdeeb and S. Ertürk, “Full depth CNN classifier for handwritten and license plate characters recognition,” *PeerJ Computer Science*, vol. 7, Art. e576, 2021.
- [8] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [9] University-UK, “EN Dataset (v4),” Roboflow Universe, CC BY 4.0, 2026. [En línea]. Disponible: <https://universe.roboflow.com/university-uk/en-t0yqi/dataset/4>
- [10] Guilhermecoleta, “Plate OCR Dataset (v4),” Roboflow Universe, CC BY 4.0, 2026. [En línea]. Disponible: <https://universe.roboflow.com/guilhermecoleta/plate-ocr-wb6jn/dataset/4>
- [11] Student-Tfetc, “LPR Character Segmentation Dataset (v3),” Roboflow Universe, CC BY 4.0, 2026. [En línea]. Disponible: <https://universe.roboflow.com/student-tfetc/lpr-character-segmentation/dataset/3>
- [12] Wilsons-Workspace-Andqz, “Modelo DeteccionPlacasEc Dataset (v3),” Roboflow Universe, CC BY 4.0, 2026. [En línea]. Disponible: [https://universe.roboflow.com/wilsons-workspace-andqz/modelo\\_deteccionplacasec/dataset/3](https://universe.roboflow.com/wilsons-workspace-andqz/modelo_deteccionplacasec/dataset/3)
- [13] ONNX Runtime developers, “ONNX Runtime,” Microsoft, 2024. [En línea]. Disponible: <https://onnxruntime.ai/>
- [14] L. A. Zadeh, “Fuzzy sets,” *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
- [15] E. H. Mamdani and S. Assilian, “An experiment in linguistic synthesis with a fuzzy logic controller,” *Int. J. Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.



- [16] T. Takagi and M. Sugeno, “Fuzzy identification of systems and its applications to modeling and control,” *IEEE Trans. Systems, Man, and Cybernetics*, vol. SMC-15, no. 1, pp. 116–132, 1985.
- [17] Presidencia de la República del Ecuador, “Reglamento a la Ley de Transporte Terrestre, Tránsito y Seguridad Vial,” Registro Oficial Suplemento 731, Ecuador, 2012, última reforma 2017.
- [18] J. Wang, C. Zhao, and Z. Liu, “Can historical accident data improve sustainable urban traffic safety? A predictive modeling study,” *Sustainability*, vol. 16, Art. no. 9642, 2024.